

BTP Progress Report: Core Infrastructure Implementation

Aditya Prakash

Roll No: 2302CS11

Department of Computer Science and Engineering, IIT Patna

September 2026

Abstract

This report details the recent progress made on the B.Tech Project titled *High-Performance Adaptive Reverse Proxy & Load Balancer*. Building upon the foundational HTTP parser and forwarder developed earlier, the project now features a complete traffic management core. The primary focus during this phase was implementing robust load balancing, automated health checking for failover, strict rate limiting, and connection pooling to improve performance. The system has been fully containerized and verified using a local Docker-based test environment.

1. Multi-Backend Load Balancing

To effectively distribute incoming traffic across multiple servers, a dynamic **LoadBalancer** component was integrated into the main event loop. The proxy reads a pool of backend addresses at startup and routes requests based on several configurable strategies:

- **Round Robin:** Simply cycles through all available and healthy backends, ensuring an even distribution of requests.
- **Weighted Round Robin:** Allows assigning different weights to servers, which is useful when backend nodes have varying compute capacities.
- **Least Connections:** Tracks the number of active requests each backend is currently handling (using thread-safe atomic counters) and routes new requests to the least busy server.
- **IP Hash:** Hashes the incoming client IP address to ensure a specific user consistently connects to the same backend, providing session stickiness.

2. Health Checking & Failover

To ensure the proxy remains reliable even when backend servers go offline, a comprehensive health monitoring system was introduced. This system operates on two fronts:

- **Active Probing:** A dedicated background thread periodically attempts to connect to each backend. If a host fails to respond within a given threshold, it is marked as offline.

- **Passive Monitoring:** The forwarder watches for real-time connection issues (like timeouts or refused connections). If a backend fails repeatedly in a short time frame, it is preemptively marked as unhealthy before the next active probe.

When a backend is marked as unhealthy, the load balancer automatically skips it. Traffic is seamlessly shifted to the remaining healthy nodes, resulting in zero-downtime failover.

3. Traffic Control & Rate Limiting

To defend against abusive traffic patterns and potential denial-of-service attempts, a **Token Bucket** rate limiting algorithm was implemented. The rate limiter operates at two levels:

- **Global Limits:** Restricts the total throughput of the proxy to prevent the underlying host from becoming overwhelmed.
- **Per-IP Limits:** Isolates individual clients to ensure fair resource allocation and stops single malicious actors from monopolizing the system.

Requests that exceed these thresholds are intercepted before reaching the load balancer, and the client receives an HTTP/1.1 429 Too Many Requests response.

4. Connection Pooling

To address the significant latency penalty of establishing new TCP connections for every HTTP request, a connection pool was built into the forwarder. Instead of closing connections after forwarding a request, the proxy retains idle keep-alive sockets in a thread-safe queue. Subsequent requests reuse these existing sockets, which drastically reduces the overhead of TCP handshakes and improves overall proxy throughput.

5. Verification and Testing

The infrastructure was validated through a series of automated Python test scripts running against a local Docker Compose cluster (1 proxy node and 3 mock backend nodes):

```
# Test traffic distribution (Confirms an even split across backends)
python scripts/test_distribution.py http://localhost:8080/

# Test failover recovery (Monitors traffic while a backend is brought down)
python scripts/test_failover.py http://localhost:8080/

# Test rate limiting (Simulates a concurrent burst to trigger 429 errors)
python scripts/test_rate_limit.py http://localhost:8080/
```

Conclusion

The implementation of the core proxy infrastructure is complete. The system is now capable of securely and efficiently routing traffic, surviving node failures, and limiting abusive behavior. These improvements lay a solid foundation for the remaining planned features, primarily the integration of an in-memory caching layer to further accelerate response times.